

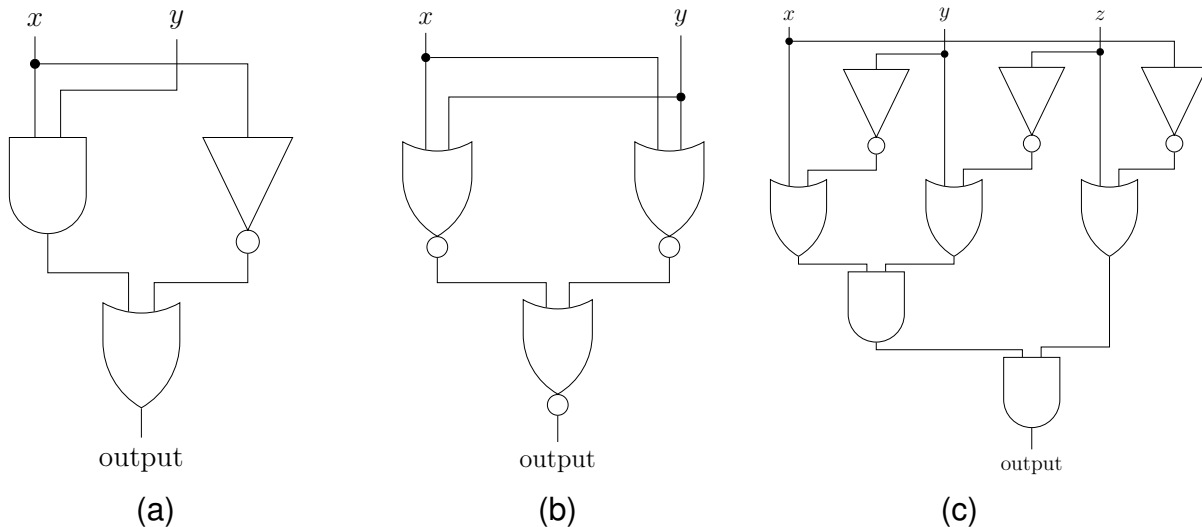
CSC173: Project 5

Boolean Logic and Boolean Circuits

We're going to keep it simple for this final end-of-term project.

We have provided code for a simple Boolean circuit simulator written in C. The download includes documentation and a small test program that demonstrates the use of the simulator API to setup a circuit and test it on some input(s). Note that this code may be different from previous terms.

Your assignment is to create and test the following circuits using the simulator framework:



DO NOT simplify the circuits or convert them into expressions.

You **MUST** turn the gates in the diagrams into the `Gates` provided in our simulator framework, and the connections between them into connections made using the function `Circuit_connect`.

For each circuit, your program **MUST** have an appropriately-named function that creates and returns an instance of a properly-configured `Circuit`.

Note that the simulator we provide does not implement `NOR` or `NAND` gates. You are welcome to extend our code and implement the other gates, or use the relevant “algebraic laws” of the Boolean Algebra to replace them. No other rewriting is permitted.

Your program **MUST** also have a **single** function that does the following:

1. Takes **any** circuit, with **any number of inputs or outputs**, as argument.
2. Runs the circuit on **all possible combinations** of its inputs.
3. Prints the resulting truth table **neatly** with:
 - (a) Input columns in **alphabetical order** followed by output columns (also alphabetical)
 - (b) Rows in order **from all inputs false (0) to all inputs true (1)**
 - (c) **Counting up in binary**
 - (d) With **both the inputs and the output(s) on a single line**, for each combination of inputs

For example:

x	y	output
0	0	1
0	1	0
1	0	0
1	1	0

Our sample code runs a few test cases. You **MUST** generalize this into a clean, well-designed, well-implemented function that can test **any** `Circuit`, with **any** number of inputs and outputs. Think about it, then read the Appendix of this document if you want some help.

Your main function **MUST** create each of the circuits by calling the appropriate function, and use the testing function described above to run them and print the results. You may print whatever other messages are required to make it clear what your program is doing (*e.g.*, what circuit is being tested).

Hint: You might find it helpful to figure out what the right answers are so that you can test your circuits automatically...

NOTE: It is possible that you may not use all the functions defined in our simulator. To avoid warnings about unused functions, add “`-Wno-unused-function`” to your `gcc` or `clang` compiler options (possibly something else for other compilers). **No other warnings may be ignored.**

Extra Credit

For up to 10% extra credit: Also implement a **single circuit** for the following pair of ternary Boolean functions f and g whose inputs are a , b , and c :

a	b	c	f	g
0	0	0	1	1
0	0	1	1	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

You **MUST** use the minterms method seen in class and in the textbook and build the circuit in three layers as seen in class and in the textbook. Test the circuit on all possible combinations of its inputs, and print the results as described previously.

Appendix follows on next page. . .

Appendix: Circuit Testing

Question: How would you test a circuit with only a single input? Think about it. . .

Answer: You need to run the circuit twice, once with the input set to 0 and once with the input set to 1. Right?

Question: How would you test a circuit with N inputs? Think about it. . .

Think about it like a Computer Scientist. . .

Answer: Pick one of the inputs. Set it to 0 and then **recursively** test with the remaining $N - 1$ inputs. Then set the chosen input to 1 and, again, **recursively** test with the remaining $N - 1$ inputs.

Question: In what order will this test the 2^N combinations of values for the N inputs? And is that the order that is required for the project?

Keep reading for additional requirements and policies. . .

Additional Requirements and Policies

The short version:

- You **must** use the following C compiler options:
`-std=c11 -Wall -Werror`
- If you are using an IDE, you **must** configure it to use those options (but I suggest that you take this opportunity to learn how to use the command-line).
- You **must** submit a ZIP including your source code and a README by the deadline.
- You **must** tell us how to build your project in your README.
- You **must** tell us how to run your project in your README.
- Projects that do not compile will receive a grade of **0**.
- Projects that do not run or that crash will receive a grade of **0** for whatever parts did not work.
- Late projects will receive a grade of **0** (see below regarding extenuating circumstances).
- You will learn the most if you do the project yourself, but collaboration is permitted in teams of up to three (3) students.
- Do not copy code from other students or from the Internet or use generative AI to do your work for you.

Detailed information follows. . .

Programming Requirements

C programs **must** be written using the “C11” dialect of C. This means using the “`-std=c11`” option with `gcc` or `clang`. For more information, see [Wikipedia](#).

You **must** also use the options “`-Wall -Werror`”. These cause the compiler to report all warnings, and to make any warnings into errors that prevent your program from compiling. You **must** be able to write code without warnings in this course.

With these settings, your program should compile and run consistently on any platform. We will deal with platform-specific discrepancies as they arise.

If you are using an IDE (Eclipse, XCode, VSCode, CLion, *etc.*), you **must** ensure that it will also build as described above. The easiest way to do that is to setup the IDE with the required compiler options. There are some notes about this in the [C Programming Resources \(for CSC173 and beyond\)](#) area.

You may **NOT** use `#pragma`'s in your programs. This includes `#pragma`'s added by an IDE or any other way that they might get into your code.

You **SHOULD** test your program with the memory checking program `valgrind`. If you don't know what `valgrind` is or why it is A Good Thing, read [C for Java Programmers Chapter 11: Debugging a C Program](#). Mac users may also want to look into the `leaks` program, which is similar.

Programs that do not receive a clean report from `valgrind` have problems that **should be fixed** whether or not the program sometimes runs properly on some platforms. The only exception is unfreed memory errors (so-called “memory leaks”). We will not penalize you for those in CSC173. However if your program does not work for us, the first thing we're going to do is run `valgrind` on it. If `valgrind` reports errors: you have errors in your program. Period.

It is easy to run `valgrind` in a virtual machine or Docker container if you cannot install and run it natively. Mac and Windows users can install [Docker Desktop](#). See [How to run Linux \(including via Docker\)](#) for more information about Linux and Docker.

For help with `valgrind`, please go to study session **well before** the project deadline.

Submission Requirements

You **must** submit your project as a ZIP archive of a folder (directory) containing the following items:

1. A file named `README.txt` or `README.pdf` (see below)
2. The source code for your project (do not include object files or executables in your submission)
3. A completed copy of the submission form posted with the project description.

The name of the folder in ZIP **must** include “CSC173”, “Project 1” (or whatever), and the NetID(s) of the submitters. For example: “CSC173_Project_1_aturing”

Your README **must** include the following information:

1. The course: “CSC173”
2. The assignment or project (e.g., “Project 1”)
3. Your name and email address
4. The names and email addresses of any collaborators (per the course policy on collaboration)
5. Instructions for building your project (with the required compiler options)
6. Instructions for running your project

The purpose of the submission form is so that we know which parts of the project you attempted and where we can find the code for some of the key required features.

- **Projects without a submission form or whose submission form does not accurately describe the project will receive a grade of 0.**
- If you cannot complete and save a PDF form, submit a text file containing the questions and your (brief) answers.

Project Evaluation

You **must** tell us in your README how to build your project and how to run it.

Note that we will NOT load projects into Eclipse or any other IDE. We **must** be able to build and run your programs from the command-line. If you have questions about that, go to a study session.

We **must** be able to cut-and-paste from your documentation in order to build and run your code. **The easier you make this for us, the better your grade will be.** It is **your** job to make the building of your project easy and the running of its program(s) easy and informative.

For C projects, the most common command for building a program from all the C source files in the directory (folder) is:

```
gcc -std=c11 -Wall -Werror -o EXECUTABLE *.c
```

where `EXECUTABLE` is the name of the executable program that we will run to execute your project.

You may also tell use to build your project using `make`. In that case, be sure to include your `Makefile` with your submission. You **must** ensure that your `Makefile` sets the compiler options appropriately.

If you expect us to do something else, you **must** describe what we need to do in your `README` file. This is unlikely to be the case for most of the projects in CSC173.

Please note that we will **NOT** under any circumstances edit your source files. That is your job.

Projects that do not compile will receive a grade of 0. There is no way to know if your program is correct solely by looking at its source code (although we can sometimes tell that is incorrect). This is actually an aspect of a very deep result in Computer Science that we cover in CSC173.

We will then run your program by running the executable, or as described in the project description. If something else is required, you **must** describe what is needed in your `README` file.

Projects that do not run or that crash will receive a grade of 0 for whatever parts did not work. You earn credit for your project by meeting the project requirements. Projects that do not run **do not** meet the requirements.

Any questions about these requirements: go to study session **BEFORE** the project is due.

Late Policy

Late projects will receive a grade of 0. You **must** submit what you have by the deadline. If there are extenuating circumstances, submit what you have before the deadline and then explain yourself via email.

If you have a medical excuse (see the course syllabus), submit what you have and explain yourself as soon as you are able.

Collaboration Policy

I assume that you are in this course to learn. You will learn the most if you do the projects **yourself**.

That said, collaboration on projects is permitted, subject to the following requirements:

- Teams of no more than 3 students, all currently taking CSC173.
- You **must** be able to explain anything you or your team submit, IN PERSON AT ANY TIME, at the instructor's or TA's discretion.
- One member of the team should submit code on the team's behalf in addition to their writeup. Other team members **must** submit a README (only) indicating who their collaborators are.
- All members of a collaborative team will get the same grade on the project.

Working in a team only works if you actually do all parts of the project together. If you only do one part of, say, three, then you only learn one third of the material. If one member of a team doesn't do their part or does it incorrectly (or dishonestly), all members pay the price.

Academic Honesty

I assume that you are in this course to learn. You will learn nothing if you do not do the projects **yourself**.

Do not copy code from other students or from the Internet.

Avoid Github and StackOverflow completely for the duration of this course.

The use of generative AI tools is **NOT** permitted in this course.

Posting homework and project solutions to public repositories on sites like GitHub is a violation of the University's Academic Honesty Policy, Section V.B.2 "Giving Unauthorized Aid." Honestly, no prospective employer wants to see your coursework. Make a great project outside of class and share that instead to show off your chops.